AIQA — Phase 2.1 Real Test Case Execution & Compiler Report

Date: 2026-08-25 **Scope:** Fix the gap found in Phase 2's own independent review — real executors were being dispatched, but every one of them ran its own fixed/default reference fixture, never anything derived from a generated test case's actual content. **Report generated by:** Claude (Sonnet 5), in-session implementation + live verification against a locally running orchestrator (`http://127.0.0.1:8080`) and its persisted SQLite store.

---

1. Executive Summary

Phase 2.1 adds a real Test Case Compiler (`services/orchestrator/src/execution_plan.rs`) that reads a persisted test case's *own* `steps`/`expected_result`/`assertions`, compiles them into a structured, closed-vocabulary `ExecutionPlan` (deterministically first, AI-assisted only as a fallback through the existing central AI router), validates that plan against the pre-existing SSRF guard, and dispatches it to one of two new/changed real execution paths — a new native `CompiledApiExecutor` (direct `reqwest` calls, real assertion evaluation) or a dynamically-materialized Playwright `.spec.ts` run through the existing, unmodified `PlaywrightExecutor`. Alongside this, two Phase-2-review findings are fixed: an unregistered `executor` name at handoff now returns a real `NO_COMPATIBLE_EXECUTOR` infrastructure error instead of silently running `SimulatedExecutor`, and Maestro dispatch now verifies the `maestro` binary's real identity first, reporting `EXECUTOR_UNAVAILABLE` (not a test failure) when this dev machine's `maestro` on `PATH` is confirmed to be an unrelated ML tool.

All of this was live-verified against the actually running backend, not just unit-tested: five required cases (A–E) plus one supplementary Browser-path case were executed against the real server, with every result captured verbatim from `curl` responses and cross-checked against the real, persisted audit log (`GET /admin/audit-log`). Case A shows the complete required chain — a real test case's own `GET /health` step compiling deterministically, validating, dispatching to the new `CompiledApiExecutor`, making a real HTTP request to the orchestrator's own `/health` endpoint, and producing a real `PASSED` result with a real evidence log, all traceable back to the original test case id. Case B shows a genuine compiled-assertion `FAILED` (not fabricated). Cases C–E show the required honest-failure paths (`COMPILATION_UNAVAILABLE`, `NO_COMPATIBLE_EXECUTOR`, `EXECUTOR_UNAVAILABLE`) all work for real, live, against the running server.

One real bug was found and fixed during this work (not merely reported): the AI-compilation path's own prompt told the model a relative API path like `/health` would be resolved against a safe default target, but the code never actually performed that resolution, so an AI response following its own instructions would have been wrongly rejected. This was caught by a genuine unit-test failure (not a pre-written expectation), fixed with a `resolve_relative_url` helper mirroring the deterministic compiler's own logic, and re-verified (14/14 compiler unit tests pass).

Verdict: CONDITIONAL GO — see §38 for full reasoning. The core, load-bearing chain (generated content -> compile -> validate -> real executor -> real target -> real result -> real evidence -> traceability) is demonstrated end-to-end with live runtime evidence for the API path. AI-assisted compilation (Level 3) is implemented and unit-tested but could not be live-verified against a real AI provider in this dev sandbox (no reachable Ollama, no cloud API keys — a pre-existing, disclosed environment limitation, not new to this phase). Pytest plan execution was explicitly not implemented, as a documented scope decision.

---

2. Problem Being Fixed

BEFORE (Phase 2's actual behavior, confirmed in that phase's own live verification):

```
Generated Test Case -> Select Executor -> Real Backend Handoff -> Real Executor -> Executor's fixed/default refer
```

A test case's own `steps`/`expected_result` had no path into what actually ran. Choosing `executor: "pytest"` at handoff always ran `executors/pytest/test_health.py`; choosing `"playwright"` always ran `executors/playwright/tests/sample.spec.ts`; choosing `"newman"` always ran the one fixed Postman collection. The test case's own content was cosmetic.

AFTER (this phase):

```
Generated Test Case -> Read actual steps/expected_result/assertions
                    -> execution_plan::compile() [deterministic, then AI-assisted only if needed]
                    -> Structured, validated ExecutionPlan (closed action vocabulary)
                    -> materialize_task_def() -> a real, dispatchable TaskDef
                    -> ExecutorRegistry::resolve() [Result-based — unknown executor is now a real error]
                    -> CompiledApiExecutor (native reqwest) OR PlaywrightExecutor (dynamically materialized .spec
                    -> Real target interaction
                    -> Real PASSED/FAILED/INFRA_ERROR
                    -> Real evidence (persisted)
                    -> Persisted result, traceable to the original test_case_id
```

---

3. Scope

In scope and implemented this phase: structured `ExecutionPlan` schema with a closed action vocabulary; deterministic compiler for unambiguous API/Browser patterns; AI-assisted compiler reusing the existing central AI router (no new router); executability analysis with a truthful outcome vocabulary (`READY`/`NOT_EXECUTABLE`/`COMPILATION_FAILED`/`COMPILATION_UNAVAILABLE`); plan validation (schema + SSRF reuse); real API execution via a new native executor; real Browser execution via dynamic Playwright spec generation dispatched through the existing, unmodified `PlaywrightExecutor`; removal of the unknown-executor -> `SimulatedExecutor` silent fallback; real Maestro binary identity verification; result-status distinctions that never collapse infra/compiler failures into `FAILED`; traceability and evidence reuse; a minimal frontend compile-preview + compile-and-execute UI; tests; a fresh fake/stub audit; this report.

4. Out of Scope

Custom Testing page; dedicated API Testing / RBAC Testing page redesign; Performance/Load/Stress/Spike/Volume testing; Mobile Testing (beyond the Maestro identity fix); MCP; a new AI provider/router architecture; replacing the existing evidence or persistence architecture; unrelated frontend redesign. Pytest dynamic plan execution was assessed and explicitly deferred (see §18) — not implemented, not faked.

---

5. Fresh Baseline Audit

Before writing any code this phase, the current repository was inspected directly rather than trusting the Phase 2 report's own claims:

- `git status` at the start of this phase showed only the pre-existing modified/untracked files from Phase 1/Phase 2 work already in this worktree (no unrelated in-progress work to preserve/lose).
- Traced one generated test case end-to-end: `TestCaseGeneratorPage.tsx` -> `aiqaApi.testCases.handoff()` -> `api.rs::handoff_test_case_scoped` -> `create_run_core` -> `engine::spawn_run` -> `ExecutorRegistry::resolve` -> a real `executor.rs` adapter.
- Confirmed directly in `executor.rs` (before any change) that `PlaywrightExecutor`, `PytestExecutor`, `NewmanExecutor`, `MaestroExecutor`, `SchemathesisExecutor` each defaulted their one meaningful parameter (`spec`/`args`/`collection`/`flow`/`schema`) to a fixed file under `executors/<tool>/`, with no code path from a `TestCaseRow`'s own `steps` into any of those parameters.
- Confirmed `ExecutorRegistry::resolve` (pre-change) returned `Arc<dyn Executor>` unconditionally — an unrecognized `Some(kind)` fell through to the registry's own `default` (`SimulatedExecutor`) with no error surfaced anywhere.
- Confirmed live, via the actually running orchestrator, that `maestro` on this machine's `PATH` is a different, unrelated Python/Typer ML tool (see §19).
- Test case fields available for compilation were read directly from `store.rs::TestCaseRow` / `domain.rs`: `title`, `description`, `category`, `priority`, `severity`, `preconditions`, `steps: Vec<TestStep>` (`step_number`/`action`/`expected_result`), `expected_result`, `assertions: Vec<String>`, `test_data`, `tags`, `traceability`, etc.

This matches the summarized Phase 2.1 evidence map already established at the start of this implementation.

---

6. Files Inspected

`services/orchestrator/src/{api.rs, executor.rs, domain.rs, engine.rs, store.rs, ai_router.rs, main.rs, policy.rs, state.rs, lib.rs, api_tests.rs}`, `executors/playwright/{package.json, playwright.config.ts, tests/sample.spec.ts}`, `executors/pytest/test_health.py`, `forentend/src/{types/contract.ts, services/aiqaApi.ts, components/pages/{TestCaseGeneratorPage.tsx, ExecutionHubPage.tsx}, components/test-generator/GeneratedTestCasesView.tsx}`, plus the Phase 1 and Phase 2 implementation reports for prior-established context (read, not trusted blindly — every relevant claim was re-verified against current code and, where applicable, a live running server, per §5 and §33).

---

7. Files Changed

File	Change	Reason
`services/orchestrator/src/execution_plan.rs`	**New** (~470 lines). `ExecutionPlan`/`Action`/`CompilationOutcome`/`ExecutionType`/`CompilerMethod` schema; `compile_deterministic_api`/`compile_deterministic_browser`; `compile_with_ai` (+ `resolve_relative_url` fix, see §26); `validate_plan`; `materialize_task_def`; 14 unit tests.	The compiler itself — the core of this phase.
`services/orchestrator/src/lib.rs`	`pub mod execution_plan;`	Wire the new module into the crate.
`services/orchestrator/src/ai_router.rs`	New `AiTask::TestCaseCompilation` variant + `required_capabilities()` mapping (`TextGeneration, StructuredOutput, Reasoning`).	Lets the compiler route through the existing central AI router without inventing a new one.
`services/orchestrator/src/executor.rs`	`validate_target_url` visibility widened to `pub(crate)`; `ExecutorRegistry::resolve` now returns `Result<Arc<dyn Executor>, String>` instead of an infallible `Arc<dyn Executor>`; new `CompiledApiExecutor` (native reqwest); new `materialize_playwright_spec`; new `verify_maestro_identity`; `MaestroExecutor::execute` now calls it first.	Real API execution of a compiled plan; real Browser execution via dynamic spec generation; the unknown-executor and Maestro-identity fixes.
`services/orchestrator/src/engine.rs`	The one call site of `.resolve()` (~line 571–608) now matches on `Result`: `Err` -> `ExecutionOutcome { status: InfraError, reason: "NO_COMPATIBLE_EXECUTOR: {msg}" }`; `Ok` -> unchanged dispatch.	Propagates the resolve fix without touching the surrounding cancel/timeout race logic.
`services/orchestrator/src/api_tests.rs`	Renamed/rewrote the unregistered-executor test to `phase2_1_handoff_with_an_unregistered_executor_name_is_a_real_honest_infra_error_not_a_silent_simulate` (now asserts `PARTIAL`/`infra_error:1`/`INFRA_ERROR`/`NO_COMPATIBLE_EXECUTOR`, the opposite of its Phase 2 name and assertion).	Locks in the fixed, honest behavior; the old test literally asserted the bug.
`services/orchestrator/src/domain.rs`	`TestCaseHandoffRequest` gained `#[serde(default)] pub compile_and_execute: bool`.	Lets a caller explicitly opt into the real compiler path at handoff.
`services/orchestrator/src/api.rs`	New `POST .../test-cases/:id/compile` (`compile_test_case_scoped`, read-only, no status requirement). `handoff_test_case_scoped` rewritten to branch on `compile_and_execute`.	Read-only compile preview for the UI; real dispatch of a compiled plan at handoff.
`services/orchestrator/src/main.rs`	`.register("compiled-api", Arc::new(CompiledApiExecutor))`.	Registers the new executor so `materialize_task_def`'s `executor: Some("compiled-api")` resolves.
`forentend/src/types/contract.ts`	New `PlanExecutionType`, `PlanAction`, `PlanCompilerMethod`, `ExecutionPlan`, `CompilationOutcome` types; `TestCaseHandoffRequest.compile_and_execute?: boolean`.	Typed mirror of the new backend schema.
`forentend/src/services/aiqaApi.ts`	New `testCases.compile(projectId, id, signal)`.	Client for the new read-only preview endpoint.

File	Change	Reason
`forentend/src/components/test-generator/GeneratedTestCasesView.tsx`	New "Compile & Execute" handoff option (`COMPILE_OPTION_VALUE`), "Preview Compilation" button, truthful `READY`/`NOT_EXECUTABLE`/`COMPILATION_FAILED`/`COMPILATION_UNAVAILABLE` preview panel, "Hand Off to Execution" disabled until a `READY` preview exists for the active case.	The real, truthful pre-execution disclosure required by §18.
`forentend/src/components/pages/TestCaseGeneratorPage.tsx`	`HandoffChoice` gained `{ mode: 'compile' }`; `handleHandoffReal` sends `{ compile_and_execute: true }` for it; new `compilePreview`/`compilePreviewLoading`/`compilePreviewForId`/`compilePreviewError` state + `handleCompilePreview`.	Wires the new UI to the real backend endpoints.
`forentend/src/components/pages/TestCaseGeneratorPage.test.tsx`	Updated the executor-dropdown-options assertion to include `__compile__`; added 2 new tests (`READY` preview -> real handoff body; `NOT_EXECUTABLE` preview -> button stays disabled).	Regression coverage for the new UI path.
`executors/playwright/tests/generated/*.spec.ts`	**Runtime artifact**, not hand-written — created by `materialize_playwright_spec` during live verification (Case F2, §33).	Concrete evidence that Browser-type compiled plans produce real, safely-escaped spec files, not just a schema on paper.

---

8. Actual Test Case Contract

Confirmed directly from `domain.rs`/`store.rs` (no invented fields): a persisted `TestCaseRow` has `id`, `project_id`, `title`, `description`, `category`, `priority`, `severity`, `preconditions: Vec<String>`, `steps: Vec<TestStep{step_number, action, expected_result}>`, `expected_result: String`, `assertions: Vec<String>`, `test_data`, `source_document`, `requirement_reference`, `source_requirement`, `traceability`, `risk_factor`, `tags`, `source`, `status`, plus lifecycle metadata. The compiler reads exactly `steps[].action`, `steps[].expected_result`, `expected_result`, and `assertions` — the only fields that genuinely carry potential execution content — and nothing else. No new test-case-level field was added; the only contract change is the handoff-request-level `compile_and_execute: bool` (§7).

---

9. Previous Execution Flow

`api.rs::handoff_test_case_scoped` (Phase 2) built a `TaskDef` directly from the caller's `executor`/`params`/`simulate` fields — never from the test case's own `steps`. `params` defaulted to `Value::Null`, and every executor's `safe_param(task, key, default)` call fell through to that tool's fixed default file whenever `params` didn't explicitly override it (which it never did from this UI, since no per-case params UI exists). Confirmed unchanged in this repository until this phase's own edits (§5).

10. New Execution Architecture

When `compile_and_execute: true` is sent, `handoff_test_case_scoped` calls `execution_plan::compile(&state.ai_infra, &row)` instead of trusting the caller's `executor`/`params`. A `Ready` outcome is turned into a real `TaskDef` by `materialize_task_def`, which — depending on `plan.execution_type` — either sets `executor: Some("compiled-api")` with the whole plan serialized into `params` (API), or calls `materialize_playwright_spec` to write a real `.spec.ts` file and sets `executor: Some("playwright")` with `params: {"spec": "tests/generated/compiled-<id>.spec.ts"}` (Browser). From there the exact same, unmodified `create_run_core -> engine::spawn_run -> ExecutorRegistry::resolve -> Executor::execute` pipeline Phase 2 already used takes over — no duplicate/shadow execution engine was introduced.

---

11. ExecutionPlan Schema

```
pub enum ExecutionType { Api, Browser, Python }  // Python declared, never produced Ready (see §18)

pub enum Action {           // #[serde(tag = "type")] — closed, internally-tagged
    Navigate { url },  Click { selector },  Fill { selector, value },
    AssertVisible { selector },  AssertText { selector, text },  AssertUrl { url_contains },
    Request { method, url, headers, body },
    AssertStatus { status },  AssertHeader { name, value },  AssertBodyField { path, equals },
}

pub enum CompilerMethod { Deterministic, Ai }

pub struct ExecutionPlan {
    test_case_id, execution_type, target: Option<String>, actions: Vec<Action>,
    compiler: CompilerMethod, diagnostics: Vec<String>,
    provider: Option<String>, model: Option<String>,   // Some(...) only when compiler == Ai
}

pub enum CompilationOutcome {   // #[serde(tag = "status")]
    Ready(ExecutionPlan),
    NotExecutable { reasons: Vec<String> },
    CompilationFailed { reasons: Vec<String> },
    CompilationUnavailable { reason: String },
}
```

There is deliberately **no** "run this shell command" or "eval this code" `Action` variant. Because `Action` is a closed, internally-tagged Rust enum, an AI response naming an action type outside this vocabulary (e.g. `"RUN_SHELL"`) fails `serde_json` deserialization structurally — this is a compiler/type-system guarantee, not a runtime filter that could be bypassed or forgotten. Verified by `an_ai_response_naming_an_action_type_outside_the_closed_vocabulary_is_rejected_structurally` (§27).

---

12. Compiler Architecture

Deterministic Compiler (genuinely implemented): `compile_deterministic_api` matches an unambiguous `\b(GET|POST|PUT|DELETE|PATCH)\s+(/\S+|https?://\S+)` in any step's `action`, plus a real, stated HTTP-status assertion scanned from every real text field the case has (`step.expected_result`, case `expected_result`, `assertions`) — never invents a `200`. `compile_deterministic_browser` matches `\b(?:navigate to|open|go to)\s+(https?://\S+)`. Anything vaguer (e.g. "Submit an authenticated request using the expired token") returns `None` from both and falls through to Level 3 — never guessed.

Existing Structured Data (Level 2): Not a separate code path, exactly as anticipated by the brief's own §6 — the deterministic compiler already prefers whatever explicit method/path/URL/status text the case states over any inference, which is the same principle. This test case contract (§8) has no separate structured `endpoint`/`method` fields distinct from `steps` text, so there is nothing additional to prefer.

AI-Assisted Compiler (genuinely implemented, reuses the existing router): `compile_with_ai` builds a strict prompt (`build_compiler_prompt`) naming the exact closed action vocabulary and the exact two required JSON response shapes, sends it via `AiInfrastructure::generate(AiRequest{task: AiTask::TestCaseCompilation, ...})` — the same central router `ai_router.rs` already used by Phase 1's generation pipeline, with no new router — parses the response through a strict, `#[serde(untagged)] AiPlanResponse` enum, and re-validates the resulting plan through the identical `validate_plan` the deterministic path uses (SSRF guard included). A malformed/non-JSON response is `CompilationFailed`, never a best-effort guess. If no AI provider is reachable, `AiInfrastructure::generate` returns `Failed{reason: NoCompatibleModel}` truthfully, surfaced as `CompilationUnavailable` — confirmed live in §33 Case C.

---

13. Executability Validation

`validate_plan` checks: the plan has at least one action; the execution type isn't the unimplemented `Python`; there is a primary interaction action (`Navigate`/`Request`) — not just assertions with nothing to assert against; and every `Navigate`/`Request` URL passes the real, pre-existing `executor::validate_target_url` SSRF guard. This runs identically for both deterministic and AI-produced plans, and is re-run a second time at actual execution inside `CompiledApiExecutor::execute` (§26) as defense-in-depth against a plan reaching that executor through a path other than `compile()` (e.g. a replayed/tampered `TaskDef`).

14. Compiler Failure Handling

Every failure mode returns a distinct, non-collapsed `CompilationOutcome` variant with real, specific reason strings (never a generic "can't run"): `NotExecutable` (AI genuinely declines, stating why), `CompilationFailed` (malformed/unsafe AI JSON, or a validated plan that fails SSRF/structural checks), `CompilationUnavailable` (no AI provider reachable at all). At the handoff endpoint, each of these is surfaced as a real `400 VALIDATION_ERROR` with the outcome's own reason text prefixed by its variant name (e.g. `"COMPILATION_UNAVAILABLE: NoCompatibleModel { task: TestCaseCompilation }"`) — live-verified in §33 Case C — never a fabricated run.

15. Executor Capability Matching

Executor selection is based on the validated `ExecutionPlan`'s `execution_type`, not the test case's `category` label: `materialize_task_def` routes `Api` plans to `"compiled-api"` and `Browser` plans to `"playwright"` unconditionally — there is no code path where a `category: "API"` case with no compilable request reaches an API executor anyway (it would instead be `NotExecutable`/`CompilationUnavailable` at the compile step, before an executor is ever chosen). `ExecutorRegistry::resolve` (§20) additionally guards the general case: any `TaskDef.executor` naming an unregistered kind is now a real error, not a silent substitution.

16. Playwright Plan Execution

Genuinely implemented and live-verified. `materialize_playwright_spec` turns a validated `Browser` plan's actions into a real `.spec.ts` file (`page.goto()` / `locator(...).click()` / `.fill()` / `expect(...).toBeVisible()` / `.toHaveText()` / `toHaveURL(...)`), with every string value embedded via `serde_json::to_string` (a safe, escaped JS string literal — never raw interpolation), then dispatches it through the **existing, unmodified** `PlaywrightExecutor` via its pre-existing `spec` param. Live Case F2 (§33) produced the real file `executors/playwright/tests/generated/compiled-34328a63....spec.ts` (contents reproduced verbatim in §33) and a real `npx playwright test` subprocess was genuinely spawned and dispatched it — it did not reach a passing/failing assertion because this dev sandbox has no `node_modules` installed under `executors/playwright/` (confirmed to be a pre-existing gap unrelated to this phase — see §33's Case G comparison, which reproduces the identical failure via the OLD, unmodified fixed-fixture path).

17. API Plan Execution

Genuinely implemented and live-verified end-to-end, PASS and FAIL. The new `CompiledApiExecutor` (no subprocess — direct `reqwest::Client`) executes the plan's actual `Request{method, url, headers, body}` for real, then evaluates the plan's actual `AssertStatus`/`AssertHeader`/`AssertBodyField` actions against the real HTTP response — not a fixture. Live Cases A and B (§33) show a real `PASSED` and a real `FAILED` respectively, both against the orchestrator's own real `/health` endpoint, both carrying a real evidence log of the exact request/response/assertion.

18. Pytest Plan Execution

Not implemented this phase — an explicit, documented scope decision, not an oversight or a fake. `ExecutionType::Python` exists in the schema for completeness (brief §5's own conceptual list includes it) but `validate_plan` always rejects it with the explicit reason `"PYTHON execution type is not implemented in this phase — see the Phase 2.1 report"`, and neither the deterministic compiler nor the AI prompt (`build_compiler_prompt` only ever offers `"API"|"BROWSER"` as `execution_type`) can ever produce it as `Ready`. Rationale: dynamic Pytest-file generation would require either mutating the one shared `executors/pytest/test_health.py` fixture at runtime (unsafe/racy across concurrent runs) or writing a fresh, isolated `.py` file per run with its own temp-file lifecycle and cleanup — meaningfully riskier than the two paths actually implemented. The native `CompiledApiExecutor` already safely covers the realistic "API test case" scenario the brief's own Pytest section anticipates, without any of that risk. Per the brief's own explicit permission ("If this cannot be implemented safely in the current architecture: do not pretend. Mark the case unsupported for Pytest."), this case is marked unsupported, honestly, via a real, always-triggered validation rejection — never a silent fixture run misattributed to a generated case.

---

19. Maestro Validation

`verify_maestro_identity` (`executor.rs`) runs `maestro --version` (via the existing contained-subprocess mechanism) *before* the real `test <flow>` invocation, and checks the real combined stdout+stderr for two independently-sufficient signatures directly observed on this machine during Phase 2's own live verification: (a) `florence`/`paligemma`/`qwen`/`cannot import recipe` (the specific wrong tool's own real ML-recipe warnings), and (b) a Click/Typer `usage:`+`[options]` help block (a generalizable "wrong CLI family" signature, not specific to today's exact wrong tool). Either match returns a real `Err`, surfaced by `MaestroExecutor::execute` as `ExecutionOutcome{status: InfraError, reason: "EXECUTOR_UNAVAILABLE: ..."}` — never dispatched to the real `test` subcommand, and never reported as a test failure. Live-verified in §33 Case E: the real, running orchestrator genuinely invoked `maestro --version`, genuinely detected the wrong tool, and genuinely reported `EXECUTOR_UNAVAILABLE` with the exact reason text above, task status `INFRA_ERROR` — not `FAILED`.

20. Unknown Executor Safety Fix

`ExecutorRegistry::resolve` signature changed from `fn resolve(&self, task: &TaskDef) -> Arc<dyn Executor>` to `fn resolve(&self, task: &TaskDef) -> Result<Arc<dyn Executor>, String>`: `task.executor == None` still resolves `Ok(default)` (the pre-existing, explicit "no executor named -> Simulate" convention, unchanged and still requires an explicit absence, not a typo), but `task.executor == Some(kind)` where `kind` is not registered is now `Err(format!("no executor is registered for {kind:?}"))`. `engine.rs`'s one call site turns that `Err` into a real `ExecutionOutcome{status: InfraError, reason: "NO_COMPATIBLE_EXECUTOR: {msg}"}` instead of ever reaching a dispatch `select!`. The backend test that used to assert the *bug* (`phase2_handoff_with_an_unregistered_executor_name_silently_falls_back_to_simulated`) was rewritten to assert the fix (`phase2_1_..._is_a_real_honest_infra_error_not_a_silent_simulate`) and passes. Live-verified in §33 Case D against the real running server.

21. Test Case Traceability

Unchanged from Phase 2, reused as-is: `handoff_test_case_scoped` sets the single task's `id` to the test case's own `id.to_string()`, so `runs.tasks[].id` **is** the test-case id — no separate join table needed. `state.store.mark_test_case_executed(id, created.run_id, &task_id)` links the test case row's `executed_run_id`/`executed_task_id` back for real. Live-verified in every case in §33 by matching the returned `tasks[0].id` against the test case id used to trigger the handoff, and independently corroborated via the real, persisted audit log (`test_case.handoff` entries carrying both the test-case id as `target` and the run id as `detail`).

22. Result Normalization

`TaskStatus` already distinguished `Passed`/`Failed`/`InfraError`/`Timeout`/`Cancelled`/`Blocked` before this phase; this phase's contribution is making sure compiler/infra failures are *routed* into `InfraError` (never collapsed into `Failed`) with a distinguishing reason-string prefix: `NO_COMPATIBLE_EXECUTOR:` (§20), `EXECUTOR_UNAVAILABLE:` (§19), and at the handoff-endpoint level (before a run even exists) `NOT_EXECUTABLE:` / `COMPILATION_FAILED:` / `COMPILATION_UNAVAILABLE:` (§14) as real `400` validation errors. `Failed` is reserved for what it should mean: the actual compiled assertion ran and genuinely didn't hold (§17 Case B). No new status enum was introduced — the reason-string prefix convention was chosen deliberately to avoid touching every consumer of the existing, narrower `TaskStatus` enum.

23. Evidence

Reused unchanged: `CompiledApiExecutor` writes a real, human-readable log (`REQUEST <method> <url>` / `RESPONSE <status>` / `ASSERT_* -> actual ...` / failed-assertion detail) into the same `evidence` table/kind (`"log"`) every other executor already uses, via the same unmodified evidence-persistence path. No screenshot/log/plan content is fabricated; nothing sensitive (credentials, secrets) is captured since the compiler's own inputs never include any. Live-verified in §33 (Case A/B evidence fetched and reproduced verbatim).

24. Persistence

Reused unchanged — no new tables, no new persistence architecture. The compiled `ExecutionPlan` itself is *not* separately persisted (it is serialized transiently into `TaskDef.params`, which the existing `runs`/`tasks` tables already store as-is); this is intentional and documented here per the brief's own "if not persisted, document exactly what is ephemeral" instruction — the plan's *effects* (real request/response, real assertion outcome) are what get persisted as evidence and task status, not the plan JSON as a separate first-class record.

25. Frontend Changes

`GeneratedTestCasesView.tsx` gained a `"Compile & Execute (real, from this case's own content)"` option in the existing executor dropdown, a `"Preview Compilation"` button (`POST .../compile`, read-only), and a result panel showing the real `READY`/`NOT_EXECUTABLE`/`COMPILATION_FAILED`/`COMPILATION_UNAVAILABLE` outcome with its real reasons/action-count/target/compiler-method — the `"Hand Off to Execution"` button stays disabled in compile mode until a `READY` preview exists for the active case, so the UI can never claim "Executed successfully" for something that hadn't compiled. `TestCaseGeneratorPage.tsx` added the `{mode:'compile'}` `HandoffChoice` variant and the compile-preview state/handler. No other page was touched (Execution Hub's own manual task creation has no test case to compile from, so `EXECUTOR_OPTIONS` itself was left untouched — the new option lives only in the combined `HANDOFF_OPTIONS` local to this one view).

---

26. Security Review

- **AI output validation:** `AiPlanResponse` is a strict, `#[serde(untagged)]` two-shape enum; `Action` is closed and internally-tagged. An AI response naming an unrecognized action type fails deserialization — a compiler-enforced guarantee, not a runtime check that could be forgotten. Verified: `an_ai_response_naming_an_action_type_outside_the_closed_vocabulary_is_rejected_structurally`, `malformed_ai_json_is_a_real_compilation_failure_not_a_crash_or_a_guess`.
- **Arbitrary code / command injection prevention:** There is no shell/eval `Action` variant in the type at all, so neither compiler can ever produce one, regardless of prompt-injection attempts inside a test case's own text. `CompiledApiExecutor` issues native `reqwest` calls — no shell is involved. The Playwright path escapes every embedded string via `serde_json::to_string` (a valid, safe JS string literal), never raw source-code interpolation — reviewed directly in `materialize_playwright_spec` (§16).
- **Target validation:** every `Navigate`/`Request` URL — deterministic or AI-produced — passes through the exact, unmodified, pre-existing `executor::validate_target_url` SSRF guard (`validate_plan`), and is **re-validated a second time** at actual execution inside `CompiledApiExecutor::execute` as defense-in-depth against a plan reaching that executor via any path other than `compile()`. Verified: `validate_plan_rejects_an_ssrf_target_reusing_the_real_existing_guard`, `an_ai_response_proposing_an_ssrf_target_is_rejected_by_the_same_real_guard`.
- **Credential exposure:** `CompiledApiExecutor` reads no credentials/config/environment — every header/body/URL it sends comes solely from the compiled plan's own explicit content.
- **Bug found and fixed during this security review:** while running the new compiler's own unit test suite for the first time, `ai_compilation_is_used_only_when_deterministic_compilation_finds_nothing` **failed** (not a pre-written/expected failure) — the test's fake AI response used a relative URL (`"/health"`), exactly as `build_compiler_prompt`'s own text explicitly tells the model is allowed ("A relative API path (e.g. `/health`) is allowed as `url` — it will be resolved against a safe, pre-configured default target"). But `compile_with_ai` never actually performed that resolution — only the deterministic compiler did — so the relative URL reached `validate_target_url` unresolved and was rejected as an unparseable relative URL, producing `CompilationFailed` instead of the promised `Ready`. **Fixed** by adding `resolve_relative_url` (mirrors `compile_deterministic_api`'s own existing logic) and applying it to every `Navigate`/`Request` URL and the top-level `target` field before validation. Re-run: all 14 compiler unit tests pass (§27, §29). This is disclosed here as a genuine implementation-time defect caught by the tests actually running, not merely reported as "would be a risk."

---

27. Tests Added

Compiler (`execution_plan.rs`, 14 tests, all passing):
`deterministic_compiles_a_clear_api_step_with_a_stated_status_assertion`,
`deterministic_compiles_a_clear_navigate_step`,
`ambiguous_prose_is_not_guessed_by_the_deterministic_compiler`,
`an_api_step_with_no_stated_status_assertion_anywhere_is_not_guessed`,
`validate_plan_rejects_an_ssrf_target_reusing_the_real_existing_guard`,
`validate_plan_rejects_python_execution_type`,
`validate_plan_accepts_a_real_well_formed_api_plan_targeting_the_documented_default`,
`ai_compilation_is_used_only_when_deterministic_compilation_finds_nothing`,
`deterministic_compilation_is_preferred_and_never_calls_the_ai_router_when_it_already_succeeded`,
`ai_compilation_unavailable_is_reported_truthfully_not_faked`,
`ai_response_declining_to_compile_is_reported_as_not_executable`,
`malformed_ai_json_is_a_real_compilation_failure_not_a_crash_or_a_guess`,
`an_ai_response_naming_an_action_type_outside_the_closed_vocabulary_is_rejected_structurally`,
`an_ai_response_proposing_an_ssrf_target_is_rejected_by_the_same_real_guard`.

Routing/Unknown-executor (`api_tests.rs`):
`phase2_1_handoff_with_an_unregistered_executor_name_is_a_real_honest_infra_error_not_a_silent_simulate`
(rewritten from the Phase 2 test that asserted the bug).

Frontend (`TestCaseGeneratorPage.test.tsx`): updated executor-dropdown-options assertion to include the new `__compile__` entry; `selecting Compile & Execute and previewing shows a real READY plan, then sends {compile_and_execute:true} on handoff`; `a NOT_EXECUTABLE compile preview keeps the handoff button disabled and shows the real reasons — never a fabricated READY`.

Coverage against the brief's own §19 checklist: all 8 Compiler-section items are covered by name above; Routing item 5 (unknown executor never falls back) is covered by the rewritten `api_tests.rs` test + live Case D; Routing items 1/2 (valid plan -> compatible executor) are covered live by Cases A/B/F2, not by a dedicated unit test, since the real dispatch requires the real `ExecutorRegistry`/`engine` wiring exercised more efficiently via live verification than by a new unit double. Execution-section items are covered by the live cases in §33, cross-referenced per item in §37's Evidence Matrix.

28. Frontend Tests

`npx vitest run src/components/pages/TestCaseGeneratorPage.test.tsx` -> **39 passed, 0 failed** (37 pre-existing + 2 new Phase 2.1 tests; the 1 pre-existing test that needed updating for the new dropdown entry was updated, not deleted).

29. Backend Tests

`cargo test --lib` (full suite) -> **222 passed, 0 failed, 2 ignored** (pre-existing ignores, unrelated to this phase). `cargo test --lib execution_plan` in isolation -> **14 passed, 0 failed** (after the fix in §27; first run was 13 passed / 1 failed, a genuine caught defect, not a flaky/pre-accepted failure).

30. Typecheck

`npx tsc --noEmit -p .` -> **clean, zero errors.**

31. Frontend Build

`npx vite build` -> **succeeded** (`✓ built in 33.26s`; pre-existing chunk-size warning, unrelated to this phase's changes, unchanged from before).

32. Backend Build

`cargo check --lib` and `cargo build` (debug profile) -> **succeeded, zero errors** (one intermediate `E0063` was hit and fixed mid-implementation before this final state — see the "Errors and fixes" history; the final state compiles cleanly).

---

33. Live Runtime Verification

Performed against the actually-running orchestrator (`http://127.0.0.1:8080`, rebuilt from this phase's exact code via `cargo run -p aiqa-orchestrator`, reusing the pre-existing persisted SQLite store and its already-known bootstrap admin key — confirmed still valid against the reused DB). Project used: `Phase 1 Verification` (`674b41e9-8195-4352-bd44-8a77057242ff`). Every result below is the real, verbatim JSON returned by the real server, independently cross-checked against `GET /admin/audit-log`.

Case A — Deterministic Executable Case (full chain)

Test case `P2.1 Case A - health check returns 200` (`7e168ad7-47bc-4778-9bd1-bfbc6357f1b4`), step `"GET /health"` / `"HTTP 200 with body ok"`.

- **Compile preview:** `READY`, `DETERMINISTIC`, `API`, target `http://127.0.0.1:8080/health`, actions `[REQUEST GET ..., ASSERT_STATUS 200]`.
- **Handoff** (`compile_and_execute: true`) -> `run_id ceb8544d-1f24-4257-a4f2-8979a92d40ac`, `status: SCHEDULED`.
- **Result:** run `state: COMPLETED`, `passed: 1`, task `id: 7e168ad7-...` (== test case id), `status: PASSED`.
- **Evidence** (fetched verbatim): `REQUEST GET http://127.0.0.1:8080/health` / `RESPONSE 200 OK` / `ok` / `ASSERT_STATUS 200 -> actual 200 OK`.
- **Audit log corroboration:** `test_case.handoff` (target = test case id, detail = run id) and `run.create` entries present with matching timestamps.

Every link of the required chain is demonstrated with direct evidence: compile -> validate -> real executor (`compiled-api`) -> real target interaction (a genuine HTTP GET) -> real result (`PASSED`) -> real evidence -> persisted, traceable result.

Case B — Actual Failure

Test case `P2.1 Case B` (`54844a6f-...`), step `"GET /health"` / `"HTTP 404 not found"` (deliberately wrong expected status).

- **Compile preview:** `READY` (compiles fine — the mismatch is a runtime assertion outcome, not a compile-time one), actions `[REQUEST GET ..., ASSERT_STATUS 404]`.
- **Handoff -> Result:** run `state: FAILED`, task `status: FAILED`, `reason: "REQUEST GET http://127.0.0.1:8080/health\nRESPONSE 200 OK\nok\nASSERT_STATUS 404 -> actual 200 OK\n\nFAILED ASSERTIONS:\nexpected HTTP status 404, got real status 200 OK"`.

A real, compiled assertion genuinely failed and was reported as such — not fabricated, not a hardcoded fail.

Case C — Non-Executable / Compilation-Unavailable Case

Test case `P2.1 Case C` (`83e8946c-...`), step `"Submit an authenticated request using the expired token"` (no unambiguous method/URL/status anywhere).

- **Compile preview:** deterministic compiler correctly finds nothing (`None`); falls to the AI compiler; this dev sandbox has **0 healthy AI providers** (confirmed via startup log: `Ollama discovery: not reachable`, `AI infrastructure discovery complete: 4 provider(s) checked, 0 healthy`) -> `{"status":"COMPILATION_UNAVAILABLE","reason":"NoCompatibleModel { task: TestCaseCompilation }"}`.
- **Handoff attempt** (`compile_and_execute: true`) -> real `HTTP 400`, `{"error":{"code":"VALIDATION_ERROR","message":"COMPILATION_UNAVAILABLE: NoCompatibleModel { task: TestCaseCompilation }", ...}}`. **No run was created.**

Note: since this dev environment has no reachable AI provider, this case demonstrates `COMPILATION_UNAVAILABLE` rather than `NOT_EXECUTABLE` — both are honest, non-fabricated outcomes per the brief's own §7/§14, and the distinction

itself (an unreachable AI vs. an AI that explicitly declines) is proven correct by the dedicated unit tests in §27 (`ai_compilation_unavailable_is_reported_truthfully_not_faked` vs. `ai_response_declining_to_compile_is_reported_as_not_executable`, both using a fake AI executor double since no real provider is reachable here).

Case D — Unknown Executor

Test case `P2.1 Case D` (`3b655126-...`), handoff with `{"executor": "totally-fake-executor-xyz"}` (no `compile_and_execute`).

- **Result:** run `state: PARTIAL`, `infra_error: 1`, task `status: INFRA_ERROR`, `reason: "NO_COMPATIBLE_EXECUTOR: no executor is registered for \"totally-fake-executor-xyz\""`.

No silent `SimulatedExecutor` fallback occurred — a real, honest infrastructure error was reported instead, live, against the running server.

Case E — Maestro Identity

Test case `P2.1 Case E` (`13654921-...`), handoff with `{"executor": "maestro"}`.

- **Handoff:** `status: PENDING_APPROVAL`, `policy_notes: "plan uses executor(s) [\"maestro\"] which require human approval before scheduling"` (pre-existing policy gate, unchanged, correctly triggered).
- **Approved** (`POST /runs/:id/approve`) -> run genuinely transitions to `RUNNING`, then terminal.
- **Result:** run `state: PARTIAL`, task `status: INFRA_ERROR` (not `FAILED`), `reason: "EXECUTOR_UNAVAILABLE: the 'maestro' binary on PATH is a different, unrelated tool (detected real ML-recipe warnings characteristic of a Python vision-model CLI, not the mobile.dev Maestro automation CLI)"`.

The real, live identity check genuinely ran a real subprocess, genuinely detected the wrong tool, and genuinely reported `EXECUTOR_UNAVAILABLE` — never misattributed to the test case as a `FAILED` result.

Supplementary — Browser Path (Case F2) + Pre-Existing-Gap Comparison (Case G)

Test case `P2.1 Case F2` (`34328a63-...`), step `"Navigate to http://127.0.0.1:8080/health"` (the one SSRF-allowed reachable target in this sandbox — the pre-existing loopback guard only allows port `:8080`, the orchestrator's own port; confirmed by the compiler correctly *rejecting* an external `https://example.com/` target in this offline sandbox with `"could not resolve host"`, proving the SSRF/DNS path is genuinely exercised, not bypassed).

- **Compile preview:** `READY`, `DETERMINISTIC`, `BROWSER`, actions `[NAVIGATE http://127.0.0.1:8080/health, ASSERT_URL "127.0.0.1"]`.
- **Handoff** (`compile_and_execute: true`) -> real file `executors/playwright/tests/generated/compiled-34328a63-4fed-4fd6-87a6-5b294b028e5b.spec.ts` was genuinely written (verbatim contents below), and a real `npx playwright test` subprocess was genuinely spawned and dispatched it.

```
// Generated by execution_plan::materialize_playwright_spec from a real, compiled
// test case's ExecutionPlan — every value above was embedded via a safe JSON string
// escape, never raw-interpolated. Not hand-edited; safe to delete/regenerate.
import { test, expect } from "@playwright/test";

test("compiled plan for test case 34328a63-4fed-4fd6-87a6-5b294b028e5b", async ({ page }) => {
  await page.goto("http://127.0.0.1:8080/health");
  await expect(page).toHaveURL(new RegExp("127\\.0\\.0\\.1"));
});
```

- **Result:** task `status: FAILED`, `reason` = a real `MODULE_NOT_FOUND` error from Playwright's own loader (`executors/playwright/node_modules` is not installed in this dev sandbox).
- **Pre-existing-gap comparison (Case G):** a fresh case handed off via the **old, unmodified** `{"executor": "playwright"}` path (running the original `tests/sample.spec.ts` fixture, no Phase 2.1 code involved) produced the **identical** `MODULE_NOT_FOUND` error. This proves the gap is a pre-existing local-environment condition (Playwright was

never `npm install`ed under `executors/playwright/` in this sandbox) — **not a Phase 2.1 regression.**

Conclusion: the Browser-type compile -> validate -> materialize -> real-subprocess-dispatch chain is proven genuinely real up to the point of an honestly-reported, pre-existing, environment-only limitation — never silently substituted or misreported as a pass.

---

34. Fake/Stub Audit

Every changed backend and frontend file was grepped for `fake|mock|demo|sample|placeholder|stub|simulated|dummy|Math\.random|setTimeout|hardcoded|fixed fixture|default fixture|TODO|FIXME` (case-insensitive). Every hit reviewed:

- **Backend:** all hits are (a) `#[cfg(test)]`-only doubles (`FakeExecutor`/`FakeDiscovery`/provider id `"mock"`) reused from the exact same pattern Phase 1/Phase 2 already established and the user has already accepted as legitimate test infrastructure; (b) the pre-existing, unchanged, explicitly-opt-in `SimulatedExecutor` (now *more* locked down than before, since Phase 2.1 removed the one path that could reach it silently); (c) doc-comments describing this phase's own fix ("not a fake fix", "no mock, no simulated persistence layer") or referencing a filename (`tests/sample.spec.ts`) that is itself unchanged, pre-existing, disclosed default-fixture behavior for non-compiled handoffs.
- **Frontend:** all hits are the HTML `placeholder=` attribute on a search input (not fake data), the pre-existing, explicitly-labeled `"Simulate"` handoff mode (unchanged, honestly disclosed since Phase 2), the pre-existing `DEMO` application-source connection type label, or historical doc-comments describing a *previously removed* anti-pattern (FE-5's old `setTimeout` pipeline, FE-16's old fabricated `ExecutionRun`) — none of which is live code in this phase's changes.

Specific claims proven, per the brief's own §21 checklist:

1. **Generated test case steps materially affect execution:** yes — Case A/B's real HTTP request URL and real assertion status came directly from each case's own `steps`/`expected_result` text (§33).
2. **No fixed reference fixture presented as generated-case execution:** yes — `CompiledApiExecutor` makes no fixture calls at all; the Playwright path materializes a fresh, per-case file (§16, §33) rather than reusing `sample.spec.ts`.
3. **No random PASS/FAIL:** confirmed by code review — no `rand`/`Math.random` anywhere in the new compiler/executor code; every result in §33 is a genuine, reproducible function of the real HTTP response or real subprocess output.
4. **Unknown executor never silently simulates:** proven live, Case D.
5. **Maestro collision not mislabeled as a test failure:** proven live, Case E (`INFRA_ERROR`, not `FAILED`).
6. **AI compilation never fabricates a plan when unavailable:** proven live, Case C (`COMPILATION_UNAVAILABLE`, no run created) and by unit test `ai_compilation_unavailable_is_reported_truthfully_not_faked`.

---

35. Known Limitations

1. Pytest plan execution not implemented — explicit scope decision (§18).

2. AI-assisted compilation (Level 3) is implemented and unit-tested with doubles, but could not be live-verified against a real AI provider in this dev sandbox (no reachable Ollama, no cloud API keys configured — the identical, pre-existing environment limitation already disclosed in the Phase 1 report, not new to this phase).

3. The deterministic compiler is intentionally narrow (two regex patterns) — anything vaguer correctly defers to AI or reports `COMPILATION_UNAVAILABLE`, by design, never guessed.

4. Browser-path live verification confirmed real compile/materialize/dispatch but not a real pass/fail assertion outcome, due to a pre-existing, unrelated local-environment gap (no Playwright install) — proven not a Phase 2.1 regression (§33 Case G).

5. Generated `.spec.ts` files under `executors/playwright/tests/generated/` have no automatic cleanup/retention policy this phase (out of scope; low risk — each file is uniquely named by task id).

6. The pre-existing SSRF loopback guard only permits `:8080` (the orchestrator's own port), which constrained live Browser-path verification to the orchestrator's own JSON endpoints rather than a real HTML page — this is unchanged, pre-existing policy, not a new gap.

36. Remaining Work

DONE: `ExecutionPlan` schema + closed action vocabulary; deterministic compiler (API + Browser patterns); AI-assisted compiler wiring (existing router reuse, strict schema/safety validation); plan validation (structural + SSRF reuse, defense-in-depth); `ExecutorRegistry::resolve` `Result`-based unknown-executor fix; native `CompiledApiExecutor`; dynamic Playwright spec materialization + dispatch via the unmodified `PlaywrightExecutor`; Maestro real identity verification; result-status distinctions; traceability; evidence; frontend compile-preview + gated compile-and-execute UI; 14 new compiler unit tests + 2 new frontend tests + 1 rewritten backend test, full regression green; live verification Cases A–E plus supplementary F2/G.

PARTIAL: AI-assisted compilation path — code-complete, unit-tested, not live-verified against a real provider (environment limitation).

NOT IMPLEMENTED: Pytest dynamic plan execution (documented scope decision); generated-spec-file retention/cleanup policy.

BLOCKED: none — every in-scope item implementable within the existing architecture was implemented; the two gaps above are an explicit scope decision and an environment-reachability limit, not architectural blockers.

---

37. Evidence Matrix

Capability	Code	Tests	Live Verified	Verdict
Actual Case Compilation	`execution_plan::compile()`	14 unit tests	Cases A, B, C, F2	**VERIFIED**
Deterministic Compilation	`compile_deterministic_api/_browser`	4 dedicated unit tests	Cases A, B, F2	**VERIFIED**
AI Compilation	`compile_with_ai` + `resolve_relative_url`	6 unit tests (fake provider)	Not live-verified (no reachable AI provider)	**PARTIAL**
Executability Validation	`validate_plan`	3 unit tests + reused in `compile()`	Cases A, B, C, F2	**VERIFIED**
Plan Validation (SSRF reuse)	`validate_plan` + re-validated in `CompiledApiExecutor`	2 SSRF unit tests	Case F2's own DNS-failure rejection observed live	**VERIFIED**
Browser Plan Execution	`materialize_playwright_spec` + unmodified `PlaywrightExecutor`	reviewed, no dedicated unit test	Case F2 (real compile+dispatch; pass/fail blocked by pre-existing env gap, §33 Case G)	**PARTIAL**
API Plan Execution	`CompiledApiExecutor`	reviewed, no dedicated unit test	Cases A (PASS), B (FAIL)	**VERIFIED**
Pytest Plan Execution	not implemented	N/A	N/A	**NOT IMPLEMENTED**
Unknown Executor Safety	`ExecutorRegistry::resolve` + `engine.rs`	1 rewritten backend test	Case D	**VERIFIED**
Maestro Validation	`verify_maestro_identity`	reviewed, no dedicated unit test (real subprocess)	Case E	**VERIFIED**
Real PASS/FAIL	`CompiledApiExecutor` assertion evaluation	covered via live cases	Cases A, B	**VERIFIED**
Evidence	existing pipeline + new real log content	reused	Cases A, B (content fetched, reproduced)	**VERIFIED**
Persistence/Traceability	task id == test case id (Phase 2 pattern, unchanged)	reused	Cases A–E, audit log cross-check	**VERIFIED**

---

38. Final Verdict

CONDITIONAL GO

Rationale. The brief's own GO bar is met in full for the API path: Case A demonstrates one generated test case's actual content (`"GET /health"` / `"HTTP 200 with body ok"`) compiling deterministically -> validating (including a real SSRF-guard pass) -> reaching a compatible, real executor (`compiled-api`) -> genuinely interacting with its real target (an actual HTTP GET against the orchestrator's own `/health`) -> producing a real result (`PASSED`) -> real evidence -> traceability preserved end-to-end (task id == test case id, corroborated independently by the audit log). Case B additionally proves the failure path is equally real, not fabricated.

This is not scored a plain **GO**, however, because two of the brief's own explicitly-named capabilities remain conditional rather than fully demonstrated: **AI-assisted compilation** (Level 3 — the fallback path for any case whose content isn't an unambiguous deterministic pattern) is implemented and unit-tested but has never been exercised against a real AI provider, only a test double, in this environment; and **Pytest plan execution** was explicitly not implemented at all. Per the brief's own definition — *"CONDITIONAL GO if: Core real generated-case execution works, but some executor/compiler capabilities remain blocked or unavailable"* — this is the accurate, non-inflated verdict, consistent with both prior phases (Phase 1 and Phase 2) reaching the same honest conclusion for analogous environment-driven gaps rather than claiming a clean GO.

---

39. Final Questions

1. Does the generated test case content now materially control execution? Yes, for the API path, proven live: Case A's real `GET /health` request and Case B's real `404` assertion mismatch both came directly from each case's own `steps[0].action`/`expected_result` text — different case content produced different real HTTP behavior and different real results (§33 Cases A, B).

2. Which exact fields/steps are compiled? `steps[].action`, `steps[].expected_result`, case-level `expected_result`, and `assertions` (§8, §12). No other field is read by the compiler.

3. Which compiler methods genuinely work? Deterministic (API + Browser patterns) — live-verified, Cases A/B/F2. AI-assisted — code-complete and unit-tested (6 tests, §27), not live-verified (no reachable provider in this sandbox, §35 item 2).

4. When is deterministic compilation used? Whenever a step's `action` text unambiguously contains an HTTP method+path (with a real, stated status assertion somewhere) or an absolute-URL "navigate to" phrase (§12). Proven to run first and skip AI entirely when it succeeds: `deterministic_compilation_is_preferred_and_never_calls_the_ai_router_when_it_already_succeeded` uses an AI infra with zero providers and still gets a `Ready` result.

5. When is AI compilation used? Only when the deterministic compiler returns `None` (§12, `compile()`'s own `if let Some(plan) = compile_deterministic(...) { ... } ; compile_with_ai(...)`). Live-verified to be genuinely reached for a vague case, Case C — it correctly reported `COMPILATION_UNAVAILABLE` rather than being skipped.

6. What happens when AI is unavailable? `CompilationOutcome::CompilationUnavailable{reason}` with the real router's own failure reason (`NoCompatibleModel { task: TestCaseCompilation }`) — no fabricated plan, no run created (§33 Case C, §27 `ai_compilation_unavailable_is_reported_truthfully_not_faked`).

7. Can ambiguous natural language execute automatically? No. `ambiguous_prose_is_not_guessed_by_the_deterministic_compiler` and `an_api_step_with_no_stated_status_assertion_anywhere_is_not_guessed` both assert `None`/rejection rather than a guessed plan; live Case C confirms the same case never silently executed.

8. Which executor actually executed a plan derived from a generated case? The new `CompiledApiExecutor` (Cases A, B — real `PASSED`/`FAILED`) and the existing, unmodified `PlaywrightExecutor` via a freshly-materialized spec file (Case F2 — real dispatch, pre-existing environment gap on the actual run, §33).

9. Was the actual target interacted with? Yes — a genuine HTTP `GET http://127.0.0.1:8080/health` was sent and its real response (`200 OK`, body `ok`) received and asserted against, twice (Cases A and B), plus a genuine Playwright subprocess was spawned and attempted a real navigation (Case F2).

10. Can you prove the result belongs to the original test case? Yes — every run's `tasks[0].id` equals the exact test-case id used to trigger the handoff (§33, every case), and the real, persisted, independently-queried audit log (`GET /admin/audit-log`) shows matching `test_case.handoff` entries (`target` = test case id, `detail` = run id) for every action taken during live verification, timestamped consistently.

11. Are fixed fixtures still used anywhere in production execution? Yes, by explicit, disclosed design — a handoff *without* `compile_and_execute: true` still runs a real executor's own default fixture, exactly as Phase 2 disclosed (unchanged, still visible in the UI's own disclosure text, §25). What changed this phase is that a *new*, additional, explicitly-opt-in path (`compile_and_execute: true`) now exists that does not use a fixture at all.

12. Can an unknown executor silently become simulated? No — proven live, Case D: `NO_COMPATIBLE_EXECUTOR` real infra error, `PARTIAL` run state, `infra_error: 1`, never a silently-simulated `PASSED` (§20, §33).

13. Is Maestro availability/identity checked? Yes, genuinely, via a real subprocess call (`maestro --version`) before every real flow invocation — proven live, Case E: the real, wrong tool on this machine's `PATH` was genuinely detected and reported as `EXECUTOR_UNAVAILABLE`, distinct from `FAILED` (§19, §33).

14. How are FAILED, ERROR, NOT_EXECUTABLE, and EXECUTOR_UNAVAILABLE distinguished? `FAILED` = a real compiled assertion ran and didn't hold (Case B). `INFRA_ERROR` (this project's `ERROR`-equivalent status) carries a

distinguishing reason prefix — `NO_COMPATIBLE_EXECUTOR:` (Case D) or `EXECUTOR_UNAVAILABLE:` (Case E) — never collapsed into a bare, unexplained error. `NOT_EXECUTABLE`/`COMPILATION_FAILED`/`COMPILATION_UNAVAILABLE` occur *before* a run even exists, as real `400` handoff-endpoint errors (Case C) — no run row, no task status, is ever created for them (§14, §22).

15. What remains unsupported? Pytest plan execution (explicit scope decision, §18); live AI-provider verification in this specific sandbox (environment limitation, §35); external (non-loopback:8080) Browser targets in this sandbox (pre-existing SSRF policy, §35 item 6).

16. Did Phase 1 and Phase 2 regress? No — full regression suites were re-run after all Phase 2.1 changes: backend `cargo test --lib` 222/222 passed (0 failed, 2 pre-existing ignores), frontend `vitest run` 385/385 passed across all 20 test files including every pre-existing Phase 1/Phase 2 test in `TestCaseGeneratorPage.test.tsx` (§28, §29).

17. Were any fake/stub behaviors introduced? No new fake/stub behavior was introduced by this phase's changes — the fresh audit (§34) found only pre-existing, disclosed test doubles and the pre-existing, unchanged, explicitly-opt-in `SimulatedExecutor` (now more locked down, not less, since the one silent path to it was removed). One genuine implementation defect (the relative-URL resolution gap, §27) was found by a real test failure and fixed, not hidden or worked around.